



Penetration Testing

Arcane 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Arcane team partnered with Hashlock to conduct a security Penetration Test of their Arcane Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Arcane is a Web3 privacy-focused infrastructure project building confidential and compliance-oriented blockchain solutions for decentralized finance and cross-chain applications. The protocol focuses on enabling private transactions, shielded asset transfers, and privacy-preserving DeFi interactions through advanced cryptographic mechanisms and decentralized infrastructure. Arcane's ecosystem includes components such as private liquidity infrastructure, privacy-enhanced wallets, cross-chain interoperability mechanisms, and institutional-grade compliance tooling designed to support secure and confidential on-chain activity across multiple blockchain ecosystems.

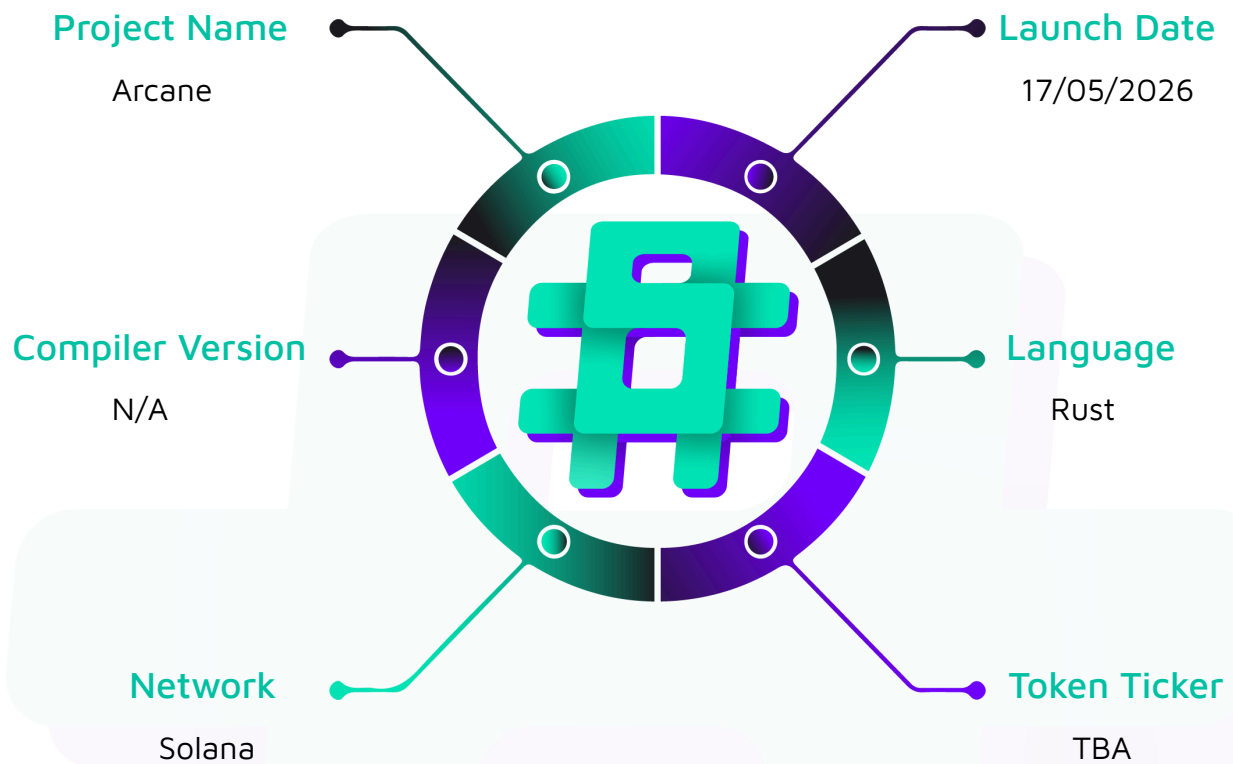
Project Name: Arcane

Project Type: DeFi

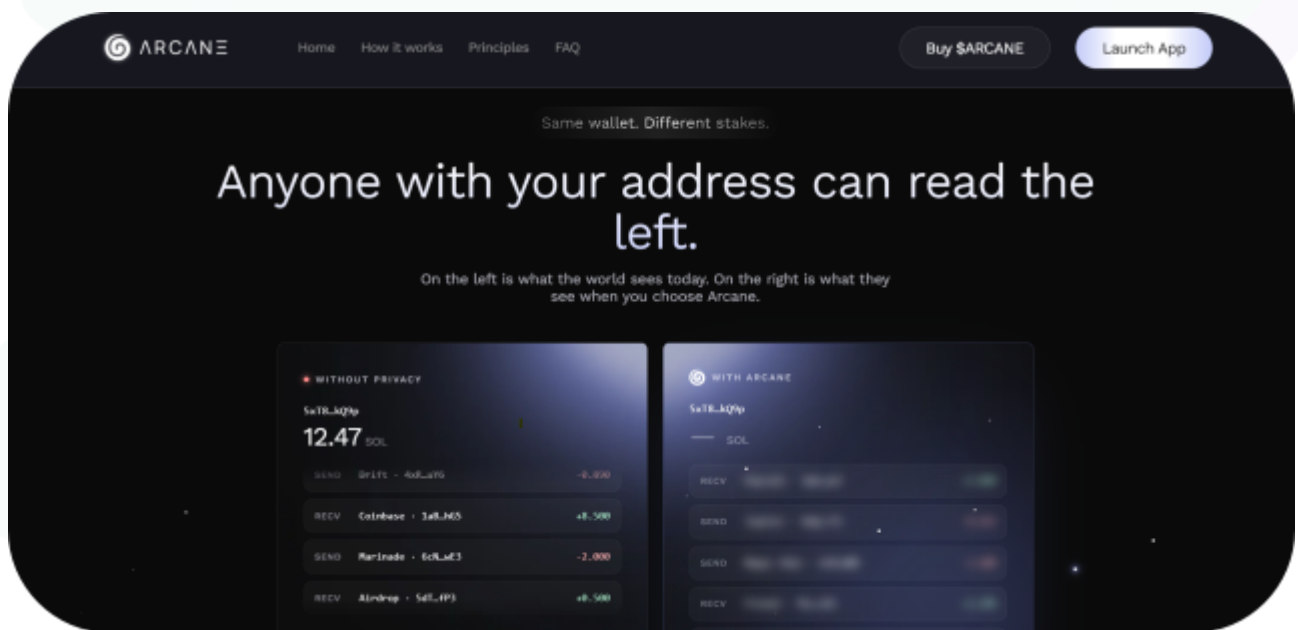
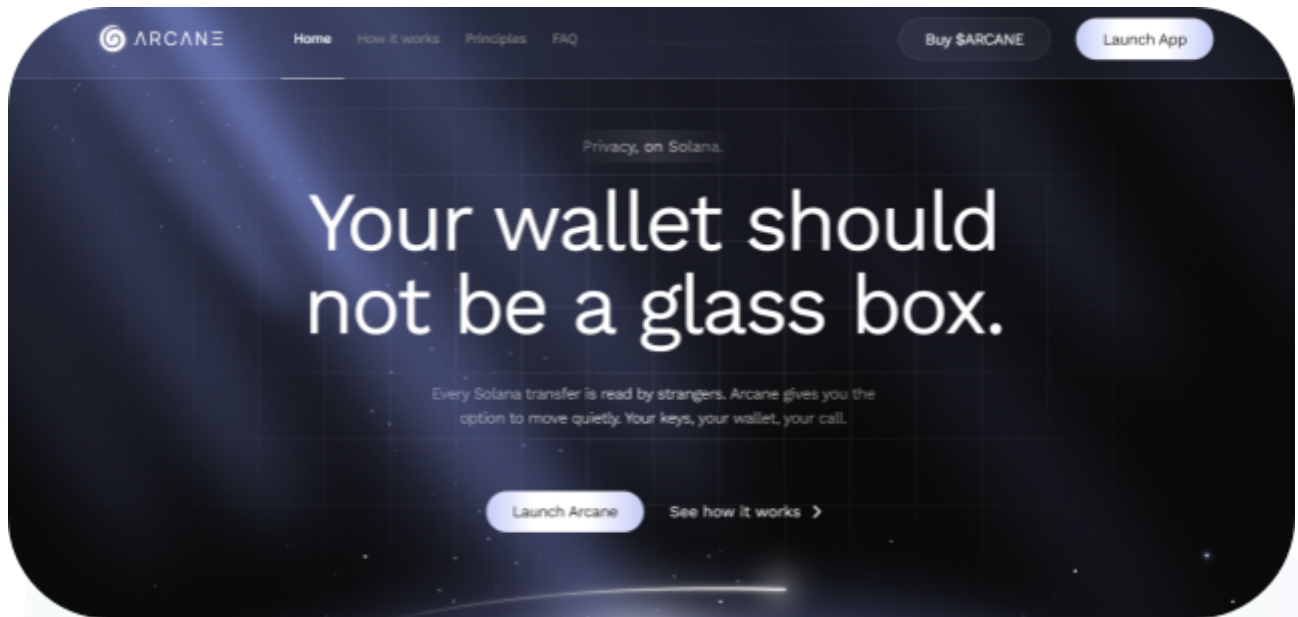
Website: <https://arcaneprivacy.com/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Arcane project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Arcane Project code base
Platform	Ethereum / Solidity/Web apps & APIs
Penetration Test Date	July, 2026
Repository/Website URL/IP Tested	https://github.com/ArcanePrivacy/Indexer
GitHub Commit Hash (If applicable)	b5af7b26a41d865f98ab99ffd1e7bc9770bec135
Fix Review GitHub Commit Hash (If applicable)	8d7eb2f56b68f78b1ccfaed0a970268aeaa06445

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High-severity vulnerabilities

2 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] [arcane-indexer] quote.js#splitAmountIntoDenominations - an uncapped withdrawal amount may lead to an unbounded loop

Description

The new `/quote` and `/review` endpoints accept a user-supplied withdrawal amount that is validated only as a positive, finite number with no upper bound, and that amount drives a synchronous loop in `splitAmountIntoDenominations` that pushes one array element per pool note. Because the iteration count grows with the requested amount and is never capped, a single unauthenticated request can freeze the event loop or exhaust process memory.

Vulnerability Details

The `parseQuoteInput` function checks the amount only for positivity and finiteness at `quote.js:95-99`, with no maximum. The value flows into `splitAmountIntoDenominations` at `quote.js:139-157`, which iterates `count = Math.floor(remaining / denomLamports)` times for each denomination and calls `splits.push` once per note. Both `POST/GET /quote` at `server.js:1723-1724` and `POST/GET /review` at `server.js:1772-1773` are unauthenticated and are not placed behind `heavyEndpointRateLimiter`. The `withQuoteTimeout` and `withReviewTimeout` wrappers use `Promise.race`, which cannot interrupt synchronous CPU-bound work, so the timeout only resolves after the loop has finished. Pool denominations come from the database and are not attacker-controlled, but the amount is.

```
// quote.js:95-99 amount validated as positive+finite only, NO maximum
const lamports = Math.round(Number(amountLamportsRaw));
if (!Number.isFinite(lamports) || lamports <= 0) {
  return { error: 'Invalid amountLamports. Must be a positive integer.' };
}
// quote.js:148-157 one push per note, iteration count is attacker-controlled
let remaining = amountLamports;
const splits = [];
for (const denomLamports of sorted) {
  const count = Math.floor(remaining / denomLamports);
  for (let i = 0; i < count; i++) {
    splits.push(denomLamports);
  }
  remaining -= count * denomLamports;
}
```

Impact

A single unauthenticated request such as `GET /quote?asset=SOL&amount=1000000000` produces roughly ten million synchronous iterations against typical pool denominations, and larger amounts scale linearly into billions of array pushes. The synchronous loop blocks the event loop and freezes the entire indexer for every other client, and a large enough amount exhausts the heap and crashes the process, giving a cheap and repeatable denial-of-service against the whole service.

Recommendation

Reject amounts above a configured maximum in `parseQuoteInput` and `parseReviewInput` before splitting, and independently cap the split count, for example by erroring out when `amountLamports / largestDenomination` exceeds `SUBMIT_MAX_PROOFS`. Compute per-denomination counts arithmetically instead of pushing one element per note, and place `/quote` and `/review` behind `heavyEndpointRateLimiter`.

Status

Resolved



Medium

[M-01] **[arcane-indexer]** **pool-realtime.js#subscribe:nullifier** -

unauthenticated Socket.io clients can join unbounded rooms

Description

The new `socket.io` channel registers `subscribe:nullifier` and `subscribe:commitment` handlers that accept an arbitrary unvalidated string and call `socket.join` on a room derived from it, with no per-socket room cap and no per-event rate limit. A single connected socket can join millions of unique rooms and grow the server room maps without bound.

Vulnerability Details

The `subscribe:nullifier` handler joins `nullifierRoom(nullifierHash)` with no format or length validation (`pool-realtime.js:105-115`), and `subscribe:commitment` behaves the same way (`pool-realtime.js:131-141`). There is no per-socket room cap, no per-event rate limit, no `maxHttpBufferSize`, and no connection cap, and the HTTP rate limiter does not cover the transport after upgrade. Separately, `parseSocketCorsOrigin` returns `true` when `SOCKET_CORS_ORIGIN` is unset (`pool-realtime.js:33-39`), reflecting any `Origin` into `Access-Control-Allow-Origin`. The unbounded room growth is a memory-exhaustion vector.

```
// pool-realtime.js:105-115 arbitrary string joined as a room, no cap or rate
limit
socket.on('subscribe:nullifier', (payload = {}, ack) => {
  const nullifierHash = payload?.nullifierHash;
  if (!nullifierHash) { /* ... */ return; }
  const room = nullifierRoom(nullifierHash);
  socket.join(room);
  if (typeof ack === 'function') ack({ ok: true, room });
});
// pool-realtime.js:33-39 default reflects ANY origin
function parseSocketCorsOrigin() {
  const raw = (process.env.SOCKET_CORS_ORIGIN || '').trim();
  if (!raw) return true;
}
```

Impact

A single unauthenticated socket can drive the indexer to memory exhaustion by joining an unbounded number of rooms, crashing the service for all clients, and any website can silently attach to the realtime deposit and withdraw feed.

Recommendation

Validate `nullifierHash` and `commitment` against their expected fixed hex length before `socket.join`, cap the number of rooms per socket and add a per-socket event rate limit, set an explicit `maxHttpBufferSize` and connection limit.

Status

Resolved

[M-02] [arcane-indexer] server.js#getClientIp - new outbound-amplifying endpoints are rate-limited by a spoofable X-Forwarded-For key

Description

The diff adds `/quote` and `/review`, which each fan out one outbound relayer `/status` request per active relayer on every call, and it adds `submitRateLimiter` and `nullifierRateLimiter`. Every one of these limiters keys its per-IP bucket on `getClientIp`, which returns the first `X-Forwarded-For` value unconditionally, so an attacker rotates the header to obtain a fresh bucket on every request and defeats the limit.

Vulnerability Details

The `getClientIp` function returns the first `X-Forwarded-For` value regardless of the `TRUST_PROXY` setting (`server.js:1037-1043`), and the rate-limit bucket key is `getClientIp(req)` combined with `req.path`. The new `/quote` and `/review` routes carry only the global limiter (`server.js:1723-1724`, `server.js:1772-1773`) and each performs a per-active-relayer outbound `/status` fetch. Because there is no account model, the per-IP limiter is the only abuse control, and once bypassed the per-request relayer fan-out becomes an amplified outbound-traffic generator. This weakness also removes the rate ceiling that the split-loop denial-of-service otherwise depends on.

```
// server.js:1037-1043 first X-Forwarded-For value trusted unconditionally
function getClientIp(req) {
  const fwd = req.headers['x-forwarded-for'];
  if (typeof fwd === 'string' && fwd.trim()) {
    return fwd.split(',')[0].trim();
  }
  return req.ip || req.socket?.remoteAddress || 'unknown';
}
```

Impact

Unlimited requests to `/quote` and `/review` produce unbounded amplified outbound relay and RPC traffic from the indexer, and the added submit and nullifier limiters provide no real protection because their per-IP key is attacker-controlled.

Recommendation

Derive the client IP from `X-Forwarded-For` only when `trust proxy` is configured, and only from the trusted right-most proxy hops, otherwise use `req.socket.remoteAddress`. Add a per-request bound on the relay fan-out so a single caller cannot amplify one request into many outbound calls.

Status

Resolved

Low

[L-01] [arcane-indexer] submit.js#waitForSignatureConfirmation - a withdrawal is reported complete at the zero-confirmation processed commitment

Description

The confirmation loop treats a transaction as successful as soon as the RPC reports the `processed` commitment, which is the weakest Solana commitment and is not cluster-confirmed. A slot that only reached `processed` can be dropped during a reorg, so the indexer can report a withdrawal as complete when it never landed.

Vulnerability Details

The confirmation branch accepts `processed`, `confirmed`, or `finalized` as success (`submit.js:1102-1108`) and the submission item is then marked `completed`. Because `processed` reflects a single node before cluster confirmation, a dropped slot leaves the indexer reporting a completed withdrawal that did not settle. There is no fund impact because on-chain settlement and the single-use `nullifier` govern the actual withdrawal.

```
// submit.js:1102-1108 'processed' (0-confirmation, reorg-able) accepted as success
const confirmation = status?.value?.confirmationStatus;
if (
  confirmation === 'processed' ||
  confirmation === 'confirmed' ||
  confirmation === 'finalized'
) {
  return !status?.value?.err;
}
```

Impact

The indexer can report a withdrawal as completed when the underlying transaction was dropped, surfacing an incorrect status to the front-end. There is no fund impact because on-chain settlement is authoritative.

Recommendation

Require at least the confirmed commitment, or re-check the indexed nullifier, before marking a submission item completed, so a dropped processed transaction is not reported as a successful withdrawal.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the Arcane project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.